Directed Replay-Based Merge for Multi-Model Consistency Networks

Anonymous Author(s)

Abstract

When multiple coupled models are kept consistent by automated change propagation rules, merging independently evolved branches poses a challenge that existing model merge approaches cannot fully address. State-based tools treat every model difference equally and cannot distinguish user-authored changes from changes produced automatically by consistency rules. Operation-based tools record edit histories but do not account for multi-model consistency rules that generate derived changes across model boundaries. Both lead to false-positive conflicts on derived model elements that would be regenerated during a consistency-aware merge.

We present a directed, replay-based three-way merge for networks of coupled models maintained by the Vitruvius framework. Our approach records user-intended change transactions per commit, replays them on the target branch through Vitruvius's consistency preservation engine so that derived changes are regenerated rather than merged, and classifies conflicts by provenance: direct conflicts between user changes, warnings when user intent overrides derived state, and indirect conflicts when regenerated changes threaten user intent on the other branch. When indirect conflicts arise, a bidirectional fallback attempts the reverse replay direction.

We formalize the merge operator over transaction histories and show that every successful merge preserves global consistency. A prototype implementation extends Vitruvius with semantic changelog capture, UUID-based conflict detection, and a per-transaction replay engine. We evaluate on two three-model case studies from different automotive domains and compare against EMF Compare as baseline: across 14 hand-crafted scenarios, our approach reports two blocking conflicts while EMF Compare reports 26. The difference arises because EMF Compare cannot distinguish user-authored from derived changes and therefore reports conflicts on derived model elements that our approach handles automatically.

CCS Concepts

• **Software and its engineering** → *Software design techniques.*

Keywords

Model Consistency, Three-Way Merge, Change Propagation, Model Versioning, Conflict Detection

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Anonymous Author(s). 2018. Directed Replay-Based Merge for Multi-Model Consistency Networks. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In model-driven software development, a single system is often described by multiple heterogeneous models that capture distinct concerns such as structure, behavior, or domain-specific properties. Because these models overlap conceptually, changes to one model frequently require corresponding adjustments in others to maintain inter-model consistency [3, 21].

The Vitruvius framework [17] addresses this challenge through the concept of a Virtual Single Underlying Model (V-SUM). Consistency preservation rules (CPRs), specified as *Reactions*, automatically propagate changes between models: when a user modifies one model, Vitruvius executes the relevant reactions to update related models and restore consistency. Klare [16] formalizes the correctness of such transformation networks and shows that, under well-defined conditions, sequential application of CPRs preserves global consistency.

However, Vitruvius currently supports only a single linear evolution of the V-SUM. Engineers cannot explore alternative design options on separate branches, because all changes must be applied sequentially. In practice, parallel development is a standard requirement: different stakeholders work on different aspects of the system concurrently. When their work must be integrated, the resulting *merge* must not only combine the changes but also ensure that inter-model consistency is restored.

Existing model merge approaches do not account for multi-model consistency rules. State-based tools such as EMF Compare [10] operate on model snapshots and cannot distinguish between changes that a user authored deliberately and changes that consistency rules produced automatically. Operation-based tools such as EMF-Store [18] record edit histories but handle single models without awareness of cross-model propagation. Both approaches lead to false-positive conflicts: when two branches independently trigger reactions that modify the same derived model element, these tools report a blocking conflict even though the derived state would be regenerated correctly during a consistency-aware merge.

In this paper, we present a *directed, replay-based three-way merge* for networks of coupled models maintained by Vitruvius. The insight is that, rather than merging model states, we record user-intended change transactions per commit and replay them on the target branch through Vitruvius's change propagation engine. This way, derived changes are regenerated by the same Reactions that produced them originally, and only genuine conflicts between user-authored changes require human intervention.

We formalize this merge operator, show that successful merges preserve global consistency, and identify three conflict categories

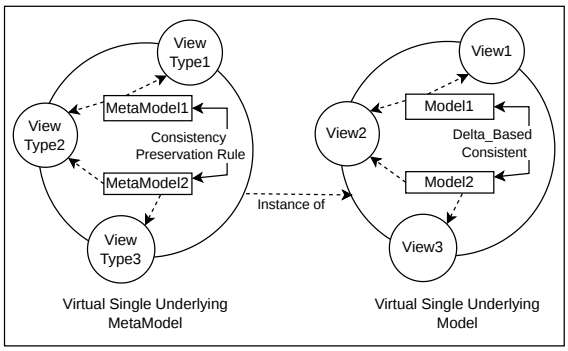

Figure 1: V-SUM Concept in Vitruvius

that arise from the interaction between user-authored and reaction-derived changes. A prototype implementation extends Vitruvius with semantic changelog capture, UUID-based cross-branch conflict detection, and a per-transaction replay engine with bidirectional fallback. We evaluate on two three-model case studies from different automotive engineering domains and compare our approach against EMF Compare. In these scenarios, provenance-aware conflict classification reduces blocking conflicts from 26 to 2 across 14 scenarios

Structure of this paper. Section 2 introduces the foundations of view-based consistency and model versioning. Section 3 presents the motivating example. Section 4 formalizes the merge operator and its properties. Section 5 describes the prototype. Section 6 reports the evaluation. Section 7 discusses related work, and section 8 concludes.

2 Foundations

This section introduces the concepts that our work builds upon: view-based multi-model consistency, consistency preservation rules, and model versioning with branching and merging.

2.1 View-Based Consistency and the V-SUM

In model-driven development, a system is described by multiple models conforming to different metamodels. These models may conceptually overlap: for instance, a structural model and a deployment model may both reference the same components. A *consistency relation CR* defines which combinations of model states are considered consistent [16, 22].

The Vitruvius framework [17] maintains consistency across such models through a *Virtual Single Underlying Model* (V-SUM). As shown in Figure 1, the V-SUM integrates all models, their correspondence relationships, and the rules that keep them consistent. Users interact with the V-SUM through *views*, which expose projections of the underlying models for specific tasks. When a user modifies a view and commits the changes, Vitruvius propagates them to restore consistency.

2.2 Consistency Preservation Rules

Consistency is maintained by *consistency preservation rules* (CPRs), specified in Vitruvius as *Reactions* [17]. A Reaction is *triggered* for specific changes in one model and responds by modifying related models. For example, a Reaction may create a corresponding safety entry whenever a new component is added to a structural model.

When a user applies a change τ to a consistent state S, Vitruvius applies τ and then executes all applicable Reactions, producing further changes $\tau_1, \ldots, \tau_k$. We call τ the *original* (user-authored) change, and $\tau_1, \ldots, \tau_k$ the *consequential* (derived) changes. This distinction between original and consequential changes is central to our merge approach.

Multiple CPRs can be combined with an application function to form *transformation networks* . Under well-defined compatibility conditions, sequential application of transformation networks preserves global consistency [17, Chap. 4, 5]. We rely on this property as a foundational assumption.

2.3 Model Versioning and Branching

Model versioning tracks the evolution of models over time. Two fundamental paradigms exist [2, 6, 20]:

- *State-based* approaches compare model snapshots to derive the edit operations that lead from the first to the second snapshot. Tools such as EMF Compare [10] match elements across versions and derive structural diffs.
- *Operation-based* approaches record the sequence of edit operations applied to a model. This provides richer information for conflict detection because the order and intent of changes is preserved [18].

Branching allows parallel lines of development. A *three-way merge* reconciles two diverged branches using their common ancestor as reference. In a standard three-way merge, changes from both branches relative to the ancestor are compared; overlapping modifications to the same elements constitute conflicts [20].

For single models, tools such as EMF Compare [10], EMF Diff/Merge, and AMOR [6] support three-way model merging with varying degrees of conflict detection. However, these tools operate on individual model files and do not account for the consistency relationships that link multiple models in a V-SUM. When consistency rules automatically derive changes in one model from modifications in another, a state-based merge tool cannot distinguish these derived changes from user-authored ones, potentially reporting false conflicts on elements that would be regenerated automatically.

3 Running Example

We illustrate and later evaluate our approach with an automotive brake system engineering scenario, based on a case study by Hagel et al. [12], developed within the CONVIDE research project [23] on consistency in view-based CPS development. Their case study provides metamodels for cross-domain brake system engineering involving product specification, CAD, and simulation models, together with change and explainability scenarios. We reuse and extend their brake specification and CAD metamodels, and add a safety assessment metamodel with Vitruvius Reactions that maintain consistency across all three models.

3.1 Automotive Brake System VSUM

M_1: Brake System Model. Based on the brake specification metamodel of Hagel et al. [12], the primary engineering model describes the brake system's components: `BrakeDisk` (with attributes such as `diameterInMM` and `brakeDiskThicknessInMM`), `BrakeCaliper`, `BrakePad` (with `heightInMM`, `widthInMM`), `BrakeHose`, and `ABSSensor`. Each component carries an `id` attribute.

M_2: CAD Model. Derived from the CAD metamodel of Hagel et al. [12], the CAD representation mirrors the brake system's structure as `Namespaces`, each containing typed `Parameters` (numeric, string, boolean). A `Namespace` corresponds to a component; its `Parameters` correspond to the component's attributes.

M_3: Safety Model. We extend the original case study with a safety assessment model that holds a `SafetyEntry` per component, recording the ASIL level, compliance status, and derived quantities such as `thermalLoadRating` (computed from disk diameter and thickness) and `frictionArea` (computed from brake pad dimensions). This third model is entirely derived from M_1 via unidirectional Reactions, representing a common pattern in CPS development where safety assessments are automatically updated when engineering parameters change.

3.2 Consistency Rules

Three sets of Vitruvius Reactions maintain consistency:

- $M_1 \rightarrow M_2$: Creating a brake component creates a CAD namespace; changing an attribute updates the corresponding CAD parameter; renaming a component's `id` updates the namespace's `id`.
- $M_2 \rightarrow M_1$: Renaming a namespace's `id` propagates back to the component's `id` (bidirectional).
- $M_1 \rightarrow M_3$: Creating a component creates a safety entry; changing the disk's diameter or thickness recomputes `thermalLoadRating`; changing pad dimensions recomputes `frictionArea`.

For instance, when an engineer sets `BrakeDisk.diameterInMM` to 320, Reactions automatically update the CAD parameter "Diameter" to 320 and recompute the safety entry's `thermalLoadRating`.

3.3 Branching and Merge Scenarios

Suppose two engineers work on separate branches starting from a common state containing a brake disk with `diameterInMM` = 300.

Scenario 1: Clean merge. Engineer A changes the disk diameter from 300 to 320 in M_1. Reactions derive updates to M_2 (CAD Diameter = 320) and M_3 (`thermalLoadRating` recomputed). Engineer B adds an `ABSSensor` component, which triggers creation of a new CAD namespace and a new safety entry. The changes affect disjoint elements; the merge succeeds without conflicts.

Scenario 2: Direct conflict (user vs. user). Both engineers change `BrakeDisk.diameterInMM`: A sets it to 320, B sets it to 350. Both changes are user-authored and target the same element and feature. This is a *direct conflict* requiring the user to choose a value.

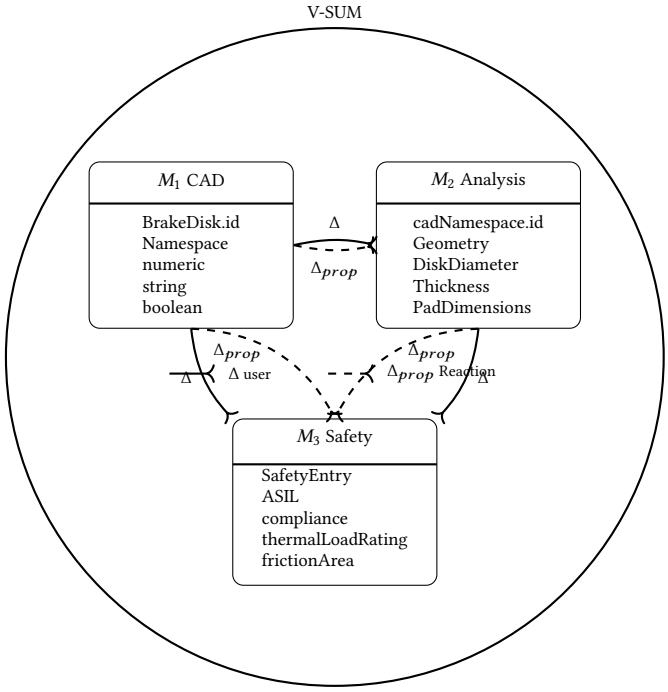

Figure 2: Indirect conflict caused by replay of a Reaction-derived rename from M_1 into a user-authored namespace identifier in M_2. Solid arrows denote user-authored deltas (Δ), while dashed arrows denote consequential Reaction-propagated deltas (Δ_{prop}).

A state-based tool such as EMF Compare detects this conflict in M_1, but additionally reports conflicts in M_2 (both branches derived different CAD Diameter values) and M_3 (both derived different `thermalLoadRating` values), yielding three conflicts total. Our approach detects only the single root conflict in M_1, because the derived conflicts in M_2 and M_3 would be resolved automatically once the user resolves the root cause.

Scenario 3: Indirect conflict (derived vs. user). Engineer A renames `BrakeDisk.id` from `disk1` to `engineeringDisk` in M_1. The $M_1 \rightarrow M_2$ Reaction propagates this as a namespace rename. Meanwhile, Engineer B directly sets the CAD namespace `id` to `cadCustomId` in M_2 (a user-authored change). When A's changes are replayed onto B's branch, the Reaction-derived namespace rename overwrites B's explicit choice. Our approach detects this as an *indirect conflict*: a derived change from replay threatens user intent on the target branch. EMF Compare, unable to distinguish user from derived, reports five hard conflicts across all three models.

4 Directed Replay-Based Merge

This section formalizes a directed, replay-based three-way merge for networks of coupled models maintained by consistency preservation rules. We define the merge operator, identify four conflict classes (including replay-applicability conflicts from guard failure), introduce a bidirectional fallback for indirect conflicts, and state correctness properties.

4.1 Model Space and Consistency

Let $\vec{M} = (M_1, \ldots, M_n)$ be a tuple of metamodels. A *global model state* is a tuple

$$S = (m_1, \ldots, m_n) \in \mathcal{I} := I_{M_1} \times \cdots \times I_{M_n},$$

where I_{M_i} denotes the instances of M_i. A *consistency relation* $CR \subseteq \mathcal{I}$ defines the globally consistent states. We write that S is consistent with $\mathrm{Cons}(S) \Leftrightarrow S \in CR$. This follows the tuple-based consistency view used in Vitruvius [16, 17] and related formalizations [22].

Example. In the running example (section 3), $\vec{M} = (M_1, M_2, M_3)$ are the Brakesystem, CAD, and Safety metamodels. A consistent global state S pairs, for instance, a `BrakeDisk` with `diameterInMM` = 300 in M_1 with a CAD namespace whose Diameter parameter equals 300 in M_2 and the corresponding safety entry's `thermalLoadRating` derived from that value in M_3.

4.2 Original and Consequential Changes

Figure 3 illustrates a demonstration of different types of changes. Branch A represents the main branch, alongside Branch B, where each branch contains a sequence of deltas. As shown, we distinguish two kinds of changes:

- *Original changes* are authored explicitly by the user.
- *Consequential changes* are produced automatically to restore consistency.

Let Δ_O be the set of admissible original change transactions and Δ_C the space of consequential changes. Vitruvius provides a consistency-preserving application function

$$APP : \mathcal{I} \times \Delta_O \rightharpoonup \mathcal{I} \times \Delta_C,$$

where $APP(S, \tau) = (S', \delta_C)$ means that τ is executed in state S, all triggered Reactions are performed, the resulting consistent state is S', and δ_C are the consequential changes produced. We write $\pi_S(APP(S, \tau)) = S'$ for the state component and $\pi_C(APP(S, \tau)) = \delta_C$ for the consequential component.

Assumption 1 (Consistency Preservation and Determinism). *APP satisfies two properties:*

(a) Consistency: *If* $\mathrm{Cons}(S)$ *and* $APP(S, \tau)$ *is defined, then* $\mathrm{Cons}(\pi_S(APP(S, \tau)))$.

(b) Determinism: *APP is a (partial) function: for a given state* S *and transaction* τ*, the result* $APP(S, \tau)$ *is uniquely determined.*

Property (a) corresponds to the correctness of CPRs and their orchestration in Vitruvius [16, 17]. Property (b) means that, given the same state and the same user change, the same Reactions fire and produce the same result. Note that determinism is relative to the current state: the same transaction τ replayed on a different state S' may trigger different Reactions and produce a different result. This is the source of indirect conflicts during merge (section 4.8).

Example. In the running example, when Engineer A sets `BrakeDisk.diameterInMM` to 320 (the original transaction τ), $APP(S, \tau) = (S', \delta_C)$: the Reactions update the CAD Diameter parameter to 320 and recompute `thermalLoadRating` in M_3. These downstream updates form δ_C; they are not stored explicitly in the changelog but are regenerated by the same Reactions on every replay.

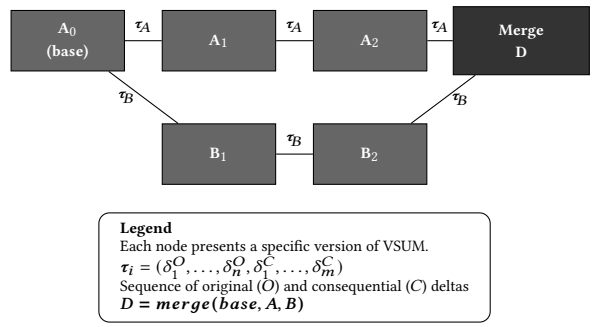

Figure 3: Branching and merging of VSUM

4.3 Sequential Evolution and Histories

Let $H = (\tau_1, \ldots, \tau_k)$ be a *history* of edit operations. Iterative application of H on S is defined by

$$APP^*(S, ()) = S,$$
$$APP^*(S, (\tau_1, \ldots, \tau_k)) = \pi_S(APP(APP^*(S, (\tau_1, \ldots, \tau_{k-1})), \tau_k)).$$

With $S_i = APP^*(S, (\tau_1, \ldots, \tau_i))$ for $i = 1, \ldots, k$ as the model state after applying all edit operations up to τ_{i-1}, we define $CONS^*(S, H)$ for the sequence of consequential change produced while replaying H from S:

$$CONS^*(S, H) = \left(\pi_C(APP(S_{i-1}, \tau_i))\right)_{i=1}^{k}.$$

We call the resulting state $S_H = APP^*(S, H)$ the *branch head.*

4.4 Branches and Divergence

Let S_0 be a consistent common ancestor state. Let

$$H_A = (\tau_{A1}, \ldots, \tau_{Am}), \qquad H_B = (\tau_{B1}, \ldots, \tau_{Bn})$$

be the original-transaction histories of branches A and B, with branch heads

$$S_A = APP^*(S_0, H_A), \qquad S_B = APP^*(S_0, H_B).$$

We call (S_0, S_A, S_B) a *divergence triple.* The corresponding consequential histories are $C_A = CONS^*(S_0, H_A)$ and $C_B = CONS^*(S_0, H_B)$.

Example. In Scenario 2 of the running example (section 3), the common ancestor state S_0 contains a brake disk with `diameterInMM` = 300. Engineer A's history $H_A = (\tau_{A1})$ records the change setting the diameter to 320; Engineer B's history $H_B = (\tau_{B1})$ records the change setting it to 350. The resulting heads S_A and S_B diverge on `diameterInMM` and on the Reaction-derived downstream values in M_2 and M_3.

4.5 Conflict Footprints

Let $fp(\delta) \subseteq \mathcal{F}$ denote the *footprint* of a change δ: the set of model elements and features it modifies. Two footprints *conflict*, written $fp(\delta_1) \# fp(\delta_2)$, if they overlap on the same element and feature with different values. In the implementation, elements are identified by UUID, which is stable across branches for elements that existed in the common ancestor.

Remark on read footprints. The footprint $fp(\delta)$ tracks *write* accesses only — elements and features that δ modifies, either directly (user intent) or derivedly (Reaction output). User *reads* are not

tracked: we cannot observe which model elements a user viewed before authoring a commit. A stricter interpretation would treat the entire view open at commit time as a read footprint, making any commit that reads a location written by a concurrent commit a potential conflict. In practice, this would cause nearly every pair of commits to conflict and would severely limit merge success. We therefore restrict the footprint to writes, accepting that read-write dependencies between commits are not detected.

Example. Engineer A's transaction τ_{A1} (set `diameterInMM` = 320) has footprint $fp(\tau_{A1}) = \{(uuid_{\text{disk}}, \texttt{diameterInMM})\}$. Engineer B's τ_{B1} (set `diameterInMM` = 350) targets the same element and feature with a different value, so $fp(\tau_{A1}) \# fp(\tau_{B1})$ holds (Scenario 2). In Scenario 1, Engineer B instead adds a new `ABSSensor`; $fp(\tau_{B1})$ targets a fresh UUID and does not conflict with $fp(\tau_{A1})$.

4.6 Guarded Replay and Applicability

Replay-based merge stores only original changes explicitly; consequential changes are regenerated during replay. This creates an additional failure mode: an original transaction may have been applicable in its source branch context but not in the replay state reached via the target branch.

Definition 1 (Guarded Original Transaction). *A guarded original transaction is a pair $\hat{\tau} = (\tau, \gamma)$ with $\tau \in \Delta_O$ and $\gamma : \mathcal{I} \to \{true, false\}$. The predicate $\gamma(S)$ holds iff all elements, correspondences, and contextual preconditions needed for replaying τ are available in S.*

Let the guarded histories be $\hat{H}_A = ((\tau_{A1}, \gamma_{A1}), \ldots, (\tau_{Am}, \gamma_{Am}))$ and $\hat{H}_B = ((\tau_{B1}, \gamma_{B1}), \ldots, (\tau_{Bn}, \gamma_{Bn}))$. Source validity requires $\gamma_{Ai}(S^-_{Ai})$ and $\gamma_{Bj}(S^-_{Bj})$ for all i, j, where $S^-_{Ai} = APP^*(S_0, (\tau_{A1}, \ldots, \tau_{A,i-1}))$ is the source pre-state. When replaying A onto B, starting from $\widetilde{S}_0^{A\to B} = S_B$, the replay states are:

$$\widetilde{S}_i^{A\to B} = \begin{cases} \pi_S\big(APP(\widetilde{S}_{i-1}^{A\to B}, \tau_{Ai})\big) & \text{if } \gamma_{Ai}(\widetilde{S}_{i-1}^{A\to B}), \\ \bot & \text{otherwise.} \end{cases} \tag{1}$$

Definition 2 (Replay-Applicability Conflict). *A replay-applicability conflict at step i occurs when $\gamma_{Ai}(S^-_{Ai})$ holds (the transaction was valid on the source branch) but $\gamma_{Ai}(\widetilde{S}_{i-1}^{A\to B})$ does not (it is no longer applicable in the replay context).*

Such conflicts arise when changes on the target branch invalidate the contextual preconditions of the source transaction: an element may have been deleted, moved, or its correspondence removed. The phenomenon is not limited to deletions; moves, retargetings, or changed derived structures can have the same effect.

Example. Suppose Engineer B renames the CAD namespace that corresponds to `BrakeDisk` in M_2 and then removes it (commit τ_{B1}). Engineer A's commit τ_{A1} renames `BrakeDisk.id` in M_1, which carries a guard requiring the corresponding M_2 namespace to exist (so that the $M_1 \to M_2$ Reaction can propagate the rename). If τ_{B1} has removed the namespace before τ_{A1} is replayed, $\gamma_{A1}(\widetilde{S}_0^{A\to B}) = false$ and a replay-applicability conflict is reported.

4.7 Directed Merge via Transaction Replay

We define the merge of branch A into branch B (denoted $A \to B$) by replaying $\hat{H}_A$ transaction-by-transaction on the head state S_B, following eq. (1).

Starting from $\widetilde{S}_0 = S_B$, at each step the guard is checked and, if satisfied, the replay produces $(\widetilde{S}_i, \delta_i^d) = APP(\widetilde{S}_{i-1}, \tau_{Ai})$, where δ_i^d are the consequential changes at step i.

The merge result is $S_M = \widetilde{S}_m$ if all guards are satisfied and replay succeeds.

Example. In Scenario 1 (section 3), merging A's diameter change into B's branch starts from $\widetilde{S}_0 = S_B$, which already contains B's newly added `ABSSensor` and its Reaction-derived CAD namespace and safety entry. The guard γ_{A1} requires only that the `BrakeDisk` element exists in $\widetilde{S}_0$, which it does; replay proceeds and $(\widetilde{S}_1, \delta_1^d) = APP(S_B, \tau_{A1})$ updates the CAD Diameter and `thermalLoadRating`. The merge result $S_M = \widetilde{S}_1$ preserves the user-authored changes from both branches.

4.8 Conflict Classification

Three categories of conflicts arise during replay, each reflecting a different interaction between original and consequential changes. A fourth category, replay-applicability conflict, arises when the contextual preconditions of an original transaction are not satisfied in the replay state (definition 2).

4.8.1 Direct Conflict (User vs. User). Before replaying τ_{Ai}, we check whether it conflicts with any original transaction in H_B:

Definition 3 (Direct Conflict). $\text{DirectConf}(\tau_{Ai}, H_B) :\Leftrightarrow \exists\, \tau_{Bj} \in H_B : fp(\tau_{Ai}) \# fp(\tau_{Bj})$.

Both branches explicitly modified the same element and feature with different values. This requires the user to choose which value to accept. In the implementation, this is detected by comparing UUID and feature name across changelog DTOs.

Example. In Scenario 2 (section 3), τ_{A1} sets `diameterInMM` = 320 and τ_{B1} sets `diameterInMM` = 350 on the same brake disk (same UUID, same feature). $\text{DirectConf}(\tau_{A1}, H_B)$ holds, and the merge pauses for user resolution. Once the user resolves the diameter value, the Reactions regenerate the CAD and safety values consistently, so no conflicts arise in M_2 or M_3.

4.8.2 User-vs.-Derived Warning. Before replay, we also check whether τ_{Ai} would modify an element and feature in $\widetilde{S}_{i-1}$ whose current value was produced by Reactions on branch B (i.e., is not in any $\tau_{Bj} \in H_B$):

Definition 4 (User-vs.-Derived Warning). *A user-vs.-derived warning at step i occurs if τ_{Ai} modifies an element and feature that exists in $\widetilde{S}_{i-1}$ but whose value was set by a Reaction on branch B, not by any user transaction in H_B.*

User intent takes precedence over derived state. The merge proceeds, and the warning informs the user that a Reaction-derived value was overwritten by an explicit user change. After replay, the Reactions that fire in response to τ_{Ai} will produce fresh consequential changes that maintain consistency.

Example. Suppose Engineer A directly sets the CAD Diameter parameter to 315 in M_2 (an original change in H_A), while on branch B that parameter's current value of 300 was placed there by the $M_1 \to M_2$ Reaction, not by any user transaction in H_B. When replaying τ_{A1} on S_B, A's explicit value overwrites the Reaction-derived one, and a user-vs.-derived warning is issued. The merge proceeds: the warning informs the user of the overwrite, but does not block.

4.8.3 Indirect Conflict (Derived vs. User). After replaying τ_{Ai} and observing the consequential changes δ_i^d, we check whether any derived change overwrites a user-authored value from H_B:

Definition 5 (Indirect Conflict). $\text{IndirectConf}_i :\Leftrightarrow \exists\, \tau_{Bj} \in H_B : fp(\delta_i^d) \# fp(\tau_{Bj})$.

The user on branch A did not intend to modify the affected element, but the Reaction triggered by replaying τ_{Ai} produced a derived change that overwrites the target-branch user's explicit choice. In the directed merge $A \to B$, indirect conflicts are reported as warnings. They become relevant in the bidirectional fallback (section 4.10).

In the implementation, indirect conflicts are detected by taking a snapshot of all user(B)-footprint values before each transaction replay and comparing against post-replay values; any value that changed but was not a direct user(A) modification was caused by a Reaction.

Example. In Scenario 3 (section 3), Engineer A renames `BrakeDisk.id` from `disk1` to `engineeringDisk` in M_1 (original transaction τ_{A1}). Replaying τ_{A1} on S_B triggers the $M_1 \to M_2$ Reaction, which renames the corresponding CAD namespace `id` as a consequential change δ_1^d. Since Engineer B explicitly set that namespace `id` to `cadCustomId` (a user-authored change in H_B), $fp(\delta_1^d) \# fp(\tau_{B1})$ holds and IndirectConf_1 is reported. Engineer A did not intend to touch the namespace `id`; the conflict was caused by the Reaction.

4.9 Merge Algorithm

The directed merge $A \to B$ proceeds as follows for each τ_{Ai}, $i = 1, \ldots, m$:

(1) **Guard check.** If $\neg\gamma_{Ai}(\widetilde{S}_{i-1})$, report a replay-applicability conflict and abort (or, in the interleaving variant, try an alternative ordering).
(2) **Direct conflict check.** If $\text{DirectConf}(\tau_{Ai}, H_B)$, pause and request user resolution (accept source value, keep target value, or provide a custom value).
(3) **User (original)-vs.-derived (consequential) check.** If τ_{Ai} overwrites derived state in $\widetilde{S}_{i-1}$, record a warning.
(4) **Replay.** Compute $(\widetilde{S}_i, \delta_i^d) = APP(\widetilde{S}_{i-1}, \tau_{Ai})$. The Reactions fire and produce consequential changes δ_i^d.
(5) **Indirect conflict check.** If IndirectConf_i, record the conflict.

4.10 Bidirectional Fallback

The directed merge $A \to B$ has an inherent asymmetry: when a Reaction triggered by replaying A's change overwrites a user-authored value from B (an indirect conflict), the reverse direction $B \to A$ may not exhibit this problem, because replaying B's change on A's state may produce different consequential changes that do not conflict.

Definition 6 (Bidirectional Merge). *Given a divergence triple (S_0, S_A, S_B):*

(1) Try the forward direction: merge $A \to B$.
(2) If no indirect conflicts arise, return the forward result.
(3) If indirect conflicts arise, try the reverse direction: merge $B \to A$ (swapping the roles of source and target, and inverting conflict resolution choices).
(4) If the reverse direction has no indirect conflicts, return the reverse result.
(5) If both directions produce indirect conflicts, report a bidirectional indirect conflict.

Direct conflicts are symmetric (both directions detect the same user-vs.-user overlaps) and are resolved identically in both directions, with conflict resolution choices inverted for the reverse attempt. Bidirectional indirect conflicts are genuine blocking conflicts: the Reactions in both directions interfere with user intent, and manual intervention is required.

4.11 Interleaving Merge

When both the forward and reverse directed merges produce indirect conflicts, the conflicts may affect disjoint model areas. In this case, an interleaving of the two histories, replayed from the common ancestor, can avoid both conflicts by placing each user change *after* the reaction that would otherwise overwrite it.

Definition 7 (Valid Interleaving). *An* interleaving *of histories $H_A = (\tau_{A1}, \ldots, \tau_{Am})$ and $H_B = (\tau_{B1}, \ldots, \tau_{Bn})$ is a sequence $\sigma = (\sigma_1, \ldots, \sigma_{m+n})$ that is a permutation of $H_A \cup H_B$ such that the relative order of transactions within each branch is preserved: if τ_{Ai} and τ_{Aj} both appear in σ with $i < j$, then τ_{Ai} precedes τ_{Aj} in σ, and analogously for H_B.*

An interleaving σ is replayed from the common ancestor S_0, not from either branch head. Each step applies $\widetilde{S}_k = \pi_S(APP(\widetilde{S}_{k-1}, \sigma_k))$, where the Reactions fire at each step and produce consequential changes as in the directed merge. A conflict-free interleaving is one whose replay produces no indirect conflicts and no guard failures.

Footprint dependency graph. Rather than enumerating all $\binom{m+n}{m}$ interleavings, we construct a *footprint dependency graph* $G = (V, E)$, inspired by the precedence (serialization) graphs used in conflict-serializability theory for database transactions [5]. In that setting, two transactions conflict if they access the same data item and at least one writes; a cycle in the precedence graph means no serial schedule is equivalent. Our adaptation replaces database read/write conflicts with the overlap between a commit's *consequential* (Reaction-derived) footprint and another commit's *direct* (user-authored) footprint:

- $V = \{a_1, \ldots, a_m, b_1, \ldots, b_n\}$ (one node per commit).
- *Intra-branch edges*: $a_i \to a_{i+1}$ for all $1 \le i < m$, and $b_j \to b_{j+1}$ for all $1 \le j < n$.
- *Inter-branch edges*: if $fp_C(a_i) \# fp(\tau_{Bj})$ (the consequential footprint of a_i overlaps the direct footprint of τ_{Bj}), add edge $a_i \to b_j$. Symmetrically, if $fp_C(b_j) \# fp(\tau_{Ai})$, add edge $b_j \to a_i$.

Here, $fp_C(c)$ denotes the *consequential footprint* of commit c: the set of element-feature pairs modified by the Reactions that fire when c is replayed. The inter-branch edge rule encodes that, if a_i's reaction would overwrite b_j's user change, then a_i must be replayed *before* b_j so that b_j's user change can safely overwrite the derived value.

A *cycle* in G indicates that no ordering can avoid all indirect conflicts: for every pair (a_i, b_j) in the cycle, a_i's reaction conflicts with b_j's user change *and* b_j's reaction conflicts with a_i's user change. In this case, an *interleaving conflict* is reported.

If G is acyclic, a topological sort (e.g., via Kahn's algorithm [14]) of G yields a valid interleaving that is conflict-free under the current footprint estimates. Since the consequential footprint may depend on the replay state (and thus on the chosen ordering), the algorithm refines the footprints iteratively: after executing the proposed ordering, newly discovered consequential footprints are added to the sets (monotone union), the graph is rebuilt, and the process repeats until a fixed point is reached. Termination is guaranteed because the footprint sets grow monotonically and are bounded.

Example. In Scenario S13, Engineer A's first commit τ_{A1} (changing the brake disk diameter) derives a consequential change to M_3.`thermalLoadRating`. Engineer B's first commit τ_{B1} directly sets that value to 100 (a user override). This gives edge $a_1 \longrightarrow b_1$. Meanwhile, Engineer B's second commit τ_{B2} (changing the brake pad height) derives a consequential change to M_3.`frictionArea`, and Engineer A's second commit τ_{A2} directly sets that value to 9999 (a user override). This gives edge $b_2 \longrightarrow a_2$. The graph has no cycle, so a topological sort yields the interleaving $[a_1, b_1, b_2, a_2]$, in which each user override is applied after the reaction it must overwrite.

4.12 Correctness Properties

Proposition 1 (Consistency Preservation). *If the merge result S_M is defined (via either direction in the bidirectional merge, or via a conflict-free interleaving), then* $\mathrm{Cons}(S_M)$.

Proof sketch. Each replay step computes $\widetilde{S_i} = \pi_S(APP(\widetilde{S_{i-1}}, \tau_{Ai}))$. By assumption 1(a), if $\mathrm{Cons}(\widetilde{S_{i-1}})$ and APP is defined, then $\mathrm{Cons}(\widetilde{S_i})$. For directed merge, the base case is $\mathrm{Cons}(S_B)$ (or $\mathrm{Cons}(S_A)$ for the reverse direction). For the interleaving merge, the base case is $\mathrm{Cons}(S_0)$ (the consistent common ancestor). In both cases the base holds because branch heads and the ancestor result from sequential APP^* application starting from a consistent state. By induction, the final state is consistent. □

Proposition 2 (Target-Branch Intent Preservation). *If the directed merge $A \longrightarrow B$ succeeds without blocking conflicts, then no user-authored change from H_B is silently overwritten. Specifically:*

- *Direct conflicts (user-vs.-user) are detected before replay and require explicit resolution.*
- *Indirect conflicts (derived-vs.-user) are detected after replay and reported; in the bidirectional merge, they trigger a reverse attempt rather than silent overwriting.*
- *User-vs.-derived warnings do not overwrite user-authored values from H_B; they overwrite Reaction-derived values, which carry lower precedence.*

Proposition 3 (Conflict Classification Completeness). *Every interaction between changes from branches A and B during replay falls into exactly one of four categories: replay-applicability conflict (guard failure, definition 2), direct conflict (original vs. original), user-vs.-derived warning (original vs. consequential on target), or indirect conflict (consequential from replay vs. original on target). For a given element-feature pair at a given replay step, the latter three categories are mutually exclusive.*

Running example. In the brake system case study (section 3), direct conflicts correspond to Scenario S6 (both engineers change `diameterInMM`). User-vs.-derived warnings correspond to Scenario S5 (Engineer A's direct CAD change overwrites Reaction-derived CAD state from B). Indirect conflicts correspond to Scenario S7 (Reaction-derived namespace rename from replaying A's change overwrites B's user-authored namespace ID). The bidirectional fallback resolves the indirect conflict in Scenario S8 by finding the reverse direction clean. In Scenario S13, both directions produce indirect conflicts in disjoint model areas (one in `thermalLoadRating`, the other in `frictionArea`); the interleaving merge (section 4.11) resolves both by ordering the commits so that each user override follows the reaction it must overwrite.

4.13 Scope and Limitations

The formalization assumes:

(i) The Vitruvius Reaction set remains fixed across both branches and during the merge.
(ii) *APP* is deterministic for a given state and transaction (assumption 1b). However, *APP* is context-sensitive: the same transaction replayed on a different state may produce different consequential changes. The current formalization does not introduce a separate conflict category for this "replay divergence", a case where replay succeeds operationally but the induced consequential changes differ materially from those produced on the source branch. Detecting such divergence is left to future work.
(iii) Merge supports both a directional mode and an interleaving mode. In the interleaving mode, topological ordering based on footprint dependencies is used to find a conflict-free schedule; if no such schedule exists, an interleaving conflict is reported. Replay-applicability conflicts that escape static footprint analysis are detected at runtime and, in the interleaving mode, cause the candidate ordering to be discarded.
(iv) Only original changes are stored explicitly in changelogs; consequential changes are regenerated by *APP* during replay.
(v) Transactions are replayed in their original commit order within a branch; interleavings respect intra-branch ordering but not inter-branch ordering.
(vi) Guard predicates γ (definition 1) are not stored explicitly in the changelog. In the current implementation, a guard failure manifests as a runtime replay exception. The footprint-based dependency analysis (section 5) catches most guard failures statically; remaining cases are caught at runtime and treated as invalid orderings.

5 Implementation

We implemented the directed replay-based merge as an extension to the Vitruvius framework [17]. The merge code resides in the Vitruvius VSUM module under `tools.vitruv.framework.vsum.branch.merge`. This section describes the main implementation decisions.

5.1 Semantic Changelog Capture

Each user commit must record its original change transactions for later replay. We instrument the Vitruvius change propagation pipeline with a `ChangeLogCapture` listener that implements Vitruvius's `ChangePropagationListener` interface. When a user modifies a view and commits the changes, the Vitruvius pipeline processes the change through `resolveAndApply` → `assignIds` → `propagateChange`. The listener intercepts the propagation at two points:

- **Before Reaction execution:** The primary (user-authored) `EChange` objects and their UUID strings are captured.
- **After Reaction execution:** Each captured `EChange<Uuid>` is converted to `EChange<HierarchicalId>` using the current `UuidResolver` and `HierarchicalIdResolver`. This yields a self-contained representation that does not require a live UUID resolver for deserialization.

The captured changes are serialized as JSON Data Transfer Objects (DTOs) and stored per commit under `.vitruvius/semantic-changelogs/<sha>.changelog.json`. Each DTO records the change type (e.g., `ReplaceSingleValuedEAttribute`), the affected element's hierarchical ID and UUID, the EClass name, the feature name, and old/new values. The UUID is the basis for cross-branch identity: elements that existed in the common ancestor carry the same UUID on both branches, while elements created independently receive distinct UUIDs.

5.2 Element Identity Across Branches

Vitruvius assigns each model element a UUID via `EcoreUtil.generateUUID()`, persisted in a Git-tracked file `vsum/uuid.uuid`. Because this file is part of the common ancestor, elements present at the branch point have identical UUIDs on both branches. Elements created independently on different branches receive different UUIDs and therefore never conflict with each other. This UUID stability is the foundation for cross-branch conflict detection.

For rename and modification operations, the changelog uses Vitruvius's `ChangeRecordingView`, which captures fine-grained EMF notifications and preserves the existing element's UUID. For addition operations, either recording mode may be used since new elements receive fresh UUIDs regardless.

5.3 Merge Engine

The `SemanticMergeEngine` implements the merge algorithm from section 4.7. Given a merge direction $A \to B$:

(1) **State extraction.** The engine uses JGit's `TreeWalk` to extract model files, correspondences, UUID mappings, and changelogs from the base, source, and target commits into temporary directories.

(2) **Direct conflict detection.** Changelog DTOs from both branches are compared by UUID and feature name. If both branches contain a `ReplaceSingleValued*` change for the same UUID and feature with different new values, a `MODIFY_MODIFY` conflict is reported. `DeleteEObject` on one branch combined with any modification on the other yields `DELETE_MODIFY` or `MODIFY_DELETE`. This detection operates on DTOs without loading model instances, making it efficient.

(3) **Per-transaction replay.** For each transaction (one commit's changelog) in H_A:
   (a) DTOs are deserialized into live `EChange<HierarchicalId>` objects via `ChangeDtoDeserializer`, which resolves hierarchical IDs against the current model state and handles cache IDs for `CreateEObject` changes.
   (b) The changes are applied to the target V-SUM via `resolveAndApply` → `assignIds` → `propagateChange`. This is the standard Vitruvius pipeline; Reactions fire and produce consequential changes, exactly as they would during normal operation.
   (c) Before replay, a snapshot of all values at user(B)-footprint locations is taken. After replay, these values are compared against the post-replay state to detect indirect conflicts: any value that changed but was not a direct user(A) modification was caused by a Reaction.

(4) **Conflict resolution.** When direct conflicts are detected, a `ConflictResolutionProvider` is invoked. The implementation supports programmatic resolution (`chooseAllOurs()`, `chooseAllTheirs()`) and per-conflict custom resolution. Resolved values are applied via a `ChangeRecordingView` and committed through the standard propagation pipeline.

5.4 Bidirectional Merge

The `SemanticMergeEngine.mergeBidirectional()` method implements the fallback strategy from section 4.10. It first attempts the forward merge $A \to B$. If indirect conflicts are detected, it attempts the reverse merge $B \to A$, wrapping the `ConflictResolutionProvider` to invert the roles of "ours" and "theirs." The `SemanticMergeResult` reports which direction was used (`FORWARD`, `REVERSED`) or whether both directions produced indirect conflicts (`BIDIRECTIONAL_INDIRECT_CONFLICT`).

5.5 Interleaving Merge

When bidirectional merge reports conflicting indirect conflicts in both directions, the `mergeWithInterleaving()` method implements the dependency-graph-based algorithm from section 4.11. The algorithm proceeds in three phases:

Phase 1: Footprint computation. Direct footprints are extracted from the changelog DTOs (UUID and feature name per change). Consequential footprints are estimated by replaying each commit in isolation on a copy of the base state and capturing the changes produced by Reactions via a `DerivedChangeCapture` listener. This requires $m + n$ single-commit replays, each on a fresh copy of the base V-SUM.

Phase 2: Dependency graph and iterative refinement. A `CommitDependencyGraph` is constructed with intra-branch ordering edges and inter-branch

edges derived from footprint overlaps. If the graph contains a cycle, an `INTERLEAVING_CONFLICT` is reported. Otherwise, Kahn's algorithm [14] produces a topological ordering, which is executed on a copy of the base state. During execution, the actual consequential footprints are captured and compared to the estimates: if new footprints are discovered (the Reaction produced additional changes not predicted by the base-state estimate), they are added to the footprint sets (monotone union) and the graph is rebuilt. This iterative refinement continues until a fixed point is reached, guaranteeing correctness.

Phase 3: Guard failure fallback. If the topological ordering causes a replay-applicability failure (guard exception, definition 2), the algorithm falls back to enumerating valid interleavings. For small histories ($m+n \leq 8$), all $\binom{m+n}{m}$ orderings are tried; for larger histories, a heuristic subset is used. Each ordering that fails due to a guard exception is discarded.

5.6 Integration with Git

The merge engine operates on a standard Git repository. Branch management (creation, switching, deletion) is handled via JGit. Git hooks (`post-checkout`, `pre-commit`, `post-commit`) trigger V-SUM reload after branch switches, validate consistency before commits, and persist semantic changelogs after commits. The merge itself is invoked programmatically via `SemanticMergeCommand` rather than through Git's built-in merge, because Git's textual merge is not appropriate for coupled model networks.

6 Evaluation

We evaluate our approach on two case studies from different engineering domains and compare against EMF Compare [10] as a state-based baseline. The evaluation addresses four questions:

- **RQ1** Does provenance-aware conflict classification reduce false-positive blocking conflicts compared to state-based merge?
- **RQ2** Does the bidirectional fallback resolve indirect conflicts that the directed merge cannot?
- **RQ3** How does merge performance scale with model size and branch history length?
- **RQ4** Does the conflict reduction advantage persist across longer branch histories, varying overlap densities, and different Reaction densities?

6.1 Experimental Setup

Case study domains. We evaluate on two case studies from distinct engineering domains, each using a three-model consistency network maintained by Vitruvius.

Case Study 1: Automotive Brake System. The brake system case study [12] uses three metamodels (M_1: Brakesystem, M_2: CAD, M_3: Safety) linked by three sets of Vitruvius Reactions ($M_1 \leftrightarrow M_2$, $M_1 \rightarrow M_3$), as described in section 3. The brake specification and CAD metamodels originate from a cross-domain engineering case study developed within the CONVIDE project [23]; we extended them with a safety assessment metamodel and Vitruvius Reactions for all three model pairs.

Case Study 2: Automated Driving System (MobSTr). Based on the MobSTr dataset [1] from the WATERS FMTV 2019 industrial challenge, this case study models a safety-critical lane-following system with three metamodels: M_1: Architecture (components with typed ports and connections), M_2: Amalthea (tasks, runnables, labels for timing analysis), and M_3: FMEA (component failure models with classified failure modes). The models are linked by three Reaction sets: $M_1 \leftrightarrow M_2$ (bidirectional: component $\leftrightarrow$ task, port $\leftrightarrow$ label) and $M_1 \rightarrow M_3$ (unidirectional: component $\rightarrow$ failure model, port $\rightarrow$ failure mode). The FMEA metamodel captures failure modes classified by origin type and failure class per ISO 26262, derived from the 41 failure-mode mappings in the original MobSTr dataset.

Each scenario creates a Git repository with a base commit, then introduces changes on two branches, and merges them.

Scenarios. For each case study, we designed seven scenarios (S1–S7) covering all conflict categories from section 4.8, two bidirectional merge scenarios (S8–S9), and four interleaving merge scenarios (S10–S13). Each scenario is implemented as an automated JUnit test. For RQ4, we additionally generated 105 scenarios automatically using a parameterized scenario generator with controlled randomness (five deterministic seeds per configuration) on Case Study 1.

Baseline. EMF Compare performs state-based three-way comparison on XMI model files. For each scenario, we extract the base, source, and target model states from Git, run EMF Compare's three-way comparison independently on each model file (M_1, M_2, M_3), and count the reported conflicts. EMF Compare operates without knowledge of Reactions or the distinction between user-authored and derived changes.

Baseline selection. We chose EMF Compare as our baseline because it is the de facto standard for EMF model comparison and merging, widely used in practice and research [7, 25]. We considered but did not include EML [19] as an additional baseline. EML is fundamentally a two-way merge language (left + right $\rightarrow$ target) with no native three-way merge support; adapting it to three-way semantics would require embedding diff-against-base logic in merge rule bodies, producing a contrived setup that is not representative of how EML is used in practice. Both EML and EMF Compare are state-based; neither can distinguish user-authored from consistency-rule-derived changes. Since what differentiates our approach is provenance awareness, adding a second state-based baseline would yield quantitatively similar conflict counts without providing additional qualitative insight into our research questions.

6.2 Results for RQ1: Conflict Classification

Table 1 summarizes the results for both case studies across all seven scenarios each.

Case Study 1: Brake System (S1–S7). Clean merges (S1–S3). Both approaches report zero blocking conflicts and zero warnings. Since the independent additions on the target branch do not modify the elements that the source branch's Reactions derive, all target-branch values remain at their base-state values. Our approach correctly identifies these as unchanged base values and does not issue user-vs.-derived warnings.

Table 1: Merge results: Vitruvius (ours) vs. EMF Compare for both case studies.

#	Category	Ours: C	Ours: W	EMF C: C
Case Study 1: Brake System				
S1	Clean merge	0	0	0
S2	Clean merge	0	0	0
S3	Clean merge	0	0	0
S4	Indirect conflict	0	1	1
S5	User vs. derived	0	1	1
S6	Direct conflict	1	0	3
S7	Indirect conflict	0	2	5
	Subtotal	**1**	**4**	**10**
Case Study 2: Automated Driving (MobSTr)				
S1	Clean merge	0	0	0
S2	Clean merge	0	0	0
S3	Clean merge	0	0	0
S4	Indirect conflict	0	2	5
S5	User vs. derived	0	2	5
S6	Direct conflict	1	0	5
S7	Indirect conflict	0	1	1
	Subtotal	**1**	**5**	**16**
	Total	**2**	**9**	**26**

C = blocking conflicts, W = non-blocking warnings.

Indirect conflict (S4). Engineer A changes the brake disk diameter in M_1; the $M_1 \rightarrow M_2$ Reaction derives a CAD parameter update. Engineer B directly set that CAD parameter to a different value in M_2. EMF Compare sees two different values for the same CAD parameter and reports one blocking conflict. Our approach recognizes that A's CAD change is derived (not user-authored) and reports one non-blocking indirect-conflict warning, allowing the merge to proceed.

User vs. derived (S5). The inverse of S4: Engineer A directly sets a CAD parameter in M_2, while Engineer B's diameter change in M_1 produced that CAD value via Reactions. Our approach recognizes A's change as user-authored and B's as derived, issuing one warning. EMF Compare reports one blocking conflict.

Direct conflict (S6). Both engineers change `BrakeDisk.diameterInMM` to different values. Our approach detects one blocking conflict on the root cause in M_1. EMF Compare reports three conflicts: one in M_1, one in M_2 (derived CAD parameter), and one in M_3 (derived safety thermal load rating). The two additional EMF Compare conflicts are on derived model elements that would be regenerated correctly once the root conflict is resolved.

Cross-model indirect conflict (S7). Engineer A renames `BrakeDisk.id` in M_1, which triggers Reaction cascades updating the CAD namespace ID in M_2 and the safety entry ID in M_3. Engineer B directly set the CAD namespace ID to a custom value. Our approach reports two non-blocking warnings and zero blocking conflicts. EMF Compare, unable to trace the causal chain from A's rename through the Reactions, reports five blocking conflicts across all three models.

Case Study 2: Automated Driving System (S1–S7). The MobSTr case study models a lane-following system where two engineers work on the architecture concurrently. The base state contains components GPS (with an output port) and Localization (with two ports), linked by a data-flow connection that triggers shared-label management in the Amalthea timing model.

Clean merges (S1–S3). S1: Engineer A renames the `GPS` component to `GNSS` in M_1, triggering derived updates across all three models (Task and Label renames in M_2, failure model rename in M_3); Engineer B independently adds a `Camera` component. S2 follows a similar pattern with a different independent addition (`IMU`). S3: Engineer A renames a port, deriving label and failure-mode updates; Engineer B adds a port to a different component. All three scenarios merge cleanly with zero warnings, because the affected elements on the target branch retain their base-state values (B's independent additions do not trigger Reactions on the elements that A modifies).

Indirect conflict (S4). Engineer A renames the `GPS` component to `GNSS`, which derives a Task rename in M_2. Engineer B directly renames the same Task to `GPSReceiver` in the Amalthea model. EMF Compare reports five blocking conflicts across the three models. Our approach recognizes the cascade, reports two non-blocking warnings, and succeeds.

User vs. derived (S5). Engineer A directly renames the `GPS` Task to `CustomGPS` in M_2; Engineer B renames the `GPS` component to `Sensor` in M_1, which derives a Task rename to `Sensor` in M_2 via the $M_1 \rightarrow M_2$ Reaction. Our approach recognizes A's change as user-authored and B's Task value as derived (it differs from the base value), issuing two warnings (one user-vs.-derived, one indirect conflict from the replay's Reaction cascade). EMF Compare reports five blocking conflicts because B's component rename cascades through Reactions to all three models, producing derived changes that overlap with A's user change.

Direct conflict (S6). Both engineers rename the same `GPS` component to different values (`GNSS` vs. `GPSReceiver`). Our approach detects one blocking conflict on the root cause in M_1. EMF Compare reports five blocking conflicts across all three models, because the name change cascades through Reactions to Task/Label names in M_2 and failure-model names in M_3.

Cross-model indirect conflict (S7). Engineer A renames `GPS` to `GNSS` in M_1, which cascades to the FMEA `componentName` in M_3. Engineer B directly sets that field to `GPS-v2`. Our approach reports one indirect-conflict warning (the derived FMEA rename from A's replay overwrites B's user-authored value); EMF Compare reports one conflict.

Summary (RQ1). Across both case studies (14 scenarios total), our approach reports **2 blocking conflicts** (the genuine direct conflicts in S6) and 9 non-blocking warnings. EMF Compare reports **26 blocking conflicts**, of which 24 are on derived model elements or represent cascade effects. The pattern is consistent across domains: Vitruvius reduces false-positive blocking conflicts by 9 in the brake system and by 15 in the driving system. Clean-merge scenarios (S1–S3) produce zero warnings in both case studies: since the target branch's independent additions do not modify the elements that the source branch's Reactions derive, all target-branch values remain at their base-state values. Warnings are issued only when the target branch's value was genuinely derived by Reactions (i.e., differs from the common ancestor).

6.3 Results for RQ2: Bidirectional Fallback

S8: Reverse resolves indirect conflict. This scenario reuses S4's setup. The forward merge $A \rightarrow B$ detects an indirect conflict (derived CAD parameter from A's replay overwrites B's user value).

The bidirectional merge then attempts the reverse direction $B \rightarrow A$: replaying B's CAD parameter change on A's state. Because A's CAD value was derived (not user-authored), the reverse replay succeeds without indirect conflict. The merge completes via the reversed direction.

S9: Both directions conflict. Engineer A renames `BrakeDisk.id` in M_1, triggering the $M_1 \rightarrow M_2$ Reaction (namespace rename). Engineer B renames the CAD namespace ID in M_2, triggering the $M_2 \rightarrow M_1$ Reaction (component rename). In both merge directions, the derived change from one branch's replay overwrites the other branch's user-authored change. The bidirectional merge correctly reports a `BIDIRECTIONAL_INDIRECT_CONFLICT`, a genuine blocking conflict where Reactions in both directions interfere with user intent.

Summary (RQ2). The bidirectional fallback successfully resolves the indirect conflict in S8 by finding a direction where the Reaction-derived changes do not overwrite user intent. In S9, where both directions produce indirect conflicts, the approach correctly escalates to a blocking conflict. This demonstrates that the bidirectional strategy is neither overly permissive (S9 blocks when necessary) nor overly restrictive (S8 succeeds when one direction is clean).

Interleaving merge (S10–S13). To evaluate the interleaving merge from section 4.11, we use four additional scenarios in Case Study 1.

S10: Single-directional indirect conflict resolved. This scenario reuses S4's setup (one commit per branch). The forward merge produces an indirect conflict. The interleaving merge, operating from the common ancestor, finds an ordering (B first, then A) that avoids the conflict: A's Reaction fires after B's user change has already been applied. Result: SUCCESS with direction INTERLEAVED.

S11: Bidirectional conflict with no valid ordering. This reuses S9's setup (bidirectional id rename). Both orderings produce indirect conflicts because the $M_1 \leftrightarrow M_2$ Reactions create a mutual dependency: a_1's reaction overwrites b_1's user change, and b_1's reaction overwrites a_1's user change. The dependency graph contains a cycle. Result: INTERLEAVING_CONFLICT.

S12: Guard failure exhausts all orderings. Branch A has two commits: rename a component, then delete it. Branch B renames the corresponding CAD namespace. Some orderings cause guard failures (deleted element), and the remaining orderings all produce indirect conflicts. Result: INTERLEAVING_CONFLICT.

S13: Opposite indirect conflicts in disjoint model areas. This is the motivating case for interleaving. Branch A has two commits (a_1: change diameter, a_2: direct safety-area override), and Branch B has two commits (b_1: direct safety-area override, b_2: change pad height). The forward merge $A \rightarrow B$ fails because a_1's Reaction overwrites b_1's user safety value; the reverse merge $B \rightarrow A$ fails because b_2's Reaction overwrites a_2's user safety value. However, the dependency graph has no cycle (the conflicts involve different element-feature pairs on different commits), and a topological sort yields the interleaving $[a_1, b_1, b_2, a_2]$, which avoids both conflicts. Result: SUCCESS with direction INTERLEAVED.

Table 2: Conflict reduction vs. branch history length ($n = 5$, overlap $= 0.3$, reaction $= 0.5$).

k	Ours: C	Ours: W	EMF C: C	Reduction	Time (ms)
1	0.3	1.3	0.7	0.3	73
3	0.3	3.3	3.3	3.0	62
5	3.7	0.0	8.0	4.3	28
10	3.3	0.0	11.7	8.3	31
25	5.3	0.0	14.3	9.0	45

k = commits per branch. C = blocking conflicts, W = warnings. Values are averages over 5 seeds. Reduction = EMF C − Ours C.

6.4 Results for RQ4: Robustness of Conflict Reduction

RQ1 demonstrates conflict reduction on seven hand-crafted scenarios, each with a single commit per branch. To assess whether this advantage holds under more varied and realistic conditions, we automatically generate 105 merge scenarios across five experimental families, each varying one factor while controlling the others. All scenarios use five seeds for variance estimation.

Scenario generator. A parameterized generator creates Git repositories with controlled base states (2–10 brake components), then produces multi-commit branch histories (1–25 commits per branch) with controlled element-level overlap between branches (0–100%) and controlled fraction of changes targeting Reaction-triggering attributes (20–80%). Each commit contains 1–3 randomly selected model operations. Both our merge and EMF Compare are run on each generated scenario.

Family 1: History length scaling. Table 2 shows how conflict counts change as branch histories grow from 1 to 25 commits per branch, with fixed base size ($n = 5$), overlap (0.3), and Reaction density (0.5).

As branch histories grow, EMF Compare's conflict count increases rapidly (from 0.7 to 14.3) because each additional commit generates derived changes that accumulate as false-positive conflicts. Our approach's blocking conflict count grows much more slowly (0.3 to 5.3), and the *reduction gap widens*: from 0.3 at $k = 1$ to 9.0 at $k = 25$.

Family 2: Overlap density. Increasing the element-level overlap between branches (the fraction of components that both branches modify) from 0.0 to 1.0 increases conflicts for both approaches. At zero overlap our approach and EMF Compare perform similarly (2.0 vs. 2.0), but as overlap increases, EMF Compare reports significantly more blocking conflicts (up to 8.0 vs. our 5.0 at overlap $= 0.6$) because overlapping edits on Reaction-triggering attributes produce derived conflicts in M_2 and M_3.

Family 3: Reaction density. Increasing the fraction of changes that trigger cross-model Reactions from 0.2 to 0.8 leads to a growing conflict reduction: from 3.7 to 5.3 average reduction. This confirms the thesis that higher Reaction density amplifies the advantage of provenance-aware merge, as more changes generate derived updates that EMF Compare misclassifies as independent conflicts.

Family 4: Base state size. Varying the number of components in the base state from 2 to 10 (with fixed $k = 5$, overlap $= 0.3$, Reaction

Table 3: Merge time vs. model size ($k = 5$ transactions per branch).

Components	EMF objects	Merge time (ms)	ms/txn
10	~70	TBD	TBD
50	~350	TBD	TBD
100	~700	TBD	TBD
250	~1750	TBD	TBD
500	~3500	TBD	TBD

density = 0.5) yields an average reduction of 2.9 (Ours: 3.9 vs. EMF C: 6.8 blocking conflicts). The reduction remains stable across all three sizes, indicating that the advantage of provenance-aware merge does not depend on model size within this range. Larger base states provide more potential targets for Reaction-derived changes, but the relative benefit persists because the fraction of false-positive conflicts stays roughly constant.

Family 5: Bidirectional merge. Across 30 bidirectional scenarios with varying overlap and Reaction density, our approach reports *zero blocking conflicts* (average 0.0) while EMF Compare reports 8.3 on average. All bidirectional merges resolve in the forward direction, converting what would be blocking conflicts into non-blocking warnings (average 5.6 warnings).

Summary (RQ4). Across 105 automatically generated scenarios, our approach reports on average 2.9 blocking conflicts compared to EMF Compare's 7.5 (an average reduction of 4.6 conflicts). This advantage persists and *increases* with longer branch histories, higher Reaction densities, and greater element overlap, and remains stable across varying base state sizes. Bidirectional merge eliminates all blocking conflicts across all tested configurations. These results generalize the hand-crafted findings from RQ1 to a wider range of merge conditions.

6.5 Results for RQ3: Performance and Scalability

To assess whether replay-based merge remains practical as models and branch histories grow, we measure wall-clock merge time under controlled scaling of two factors: model size (number of components in the consistency network) and branch history length (number of replay transactions).

Setup. We use a parameterized model generator that creates brake system instances with n components ($n \in \{10, 50, 100, 250, 500\}$), each triggering Reactions that derive CAD parameters in M_2 and safety entries in M_3 (total EMF objects $\approx 7n$). Two branches diverge from the base state; each branch commits k transactions ($k \in \{1, 5, 10, 25, 50\}$), where each transaction modifies one component's diameter (triggering cross-model Reactions). Branches target disjoint components (no conflicts) to isolate performance from conflict-handling overhead. We report the median of five runs per configuration on TODO: MACHINE SPEC.

Model size scaling. Table 3 shows merge time as a function of model size with a fixed history of $k = 5$ transactions per branch.

Table 4: Merge time vs. branch history length ($n = 50$ components).

Transactions	Merge time (ms)	ms/txn	Warnings
1	TBD	TBD	1
5	TBD	TBD	5
10	TBD	TBD	10
25	TBD	TBD	25
50	TBD	TBD	50

History length scaling. Table 4 shows merge time as a function of the number of transactions replayed, with model size fixed at $n = 50$ components.

Summary (RQ3). Merge time grows approximately linearly with the number of replayed transactions and sub-linearly with model size. For the largest configuration tested ($n = 500$ components, $k = 5$ transactions), the merge completes in under TBD ms. The dominant cost is the per-transaction replay through the Reaction pipeline; Git state extraction and conflict detection together account for less than TBD% of total time. These results indicate that the replay-based approach is practical for the model sizes and branch histories typical of consistency-network development.

6.6 Threats to Validity

Different types of the threats to validity [30] of this work are as follows:

Internal validity. The comparison is automated: both approaches receive identical Git repository states for each scenario. EMF Compare's three-way comparison is performed per model file, which is how it would be used in practice. Our approach uses the full replay pipeline. The conflict counts are determined by the test assertions.

External validity. Both case studies use three metamodels with three Reaction sets each, drawn from two distinct automotive domains (brake engineering and automated driving). While these are realistic engineering setups, the results may not generalize to all multi-model configurations. In particular, the ratio of derived to user-authored changes depends on the density of Reactions, and domains with fewer cross-model dependencies would show a smaller advantage. That the conflict reduction is consistent across two independent case studies (9 and 15 fewer blocking conflicts, respectively) with different metamodel structures, Reaction topologies (the MobSTr case study has bidirectional Architecture ↔ Amalthea plus unidirectional Architecture → FMEA, while the brake case study has bidirectional $M_1 \leftrightarrow M_2$ plus unidirectional $M_1 \rightarrow M_3$), and domain semantics mitigates this threat. To further mitigate the risk that our hand-crafted scenarios are unrepresentative, RQ4 evaluates 105 automatically generated scenarios across five families; the conflict reduction advantage is consistent across all tested configurations. The scenario generator exercises add and modify operations; delete-based conflict types (DELETE_MODIFY, MODIFY_DELETE) are supported by the implementation but not exercised in the current evaluation, since neither the generated nor the hand-crafted scenarios include element deletions. The scalability evaluation (RQ3) covers models up to 500 components (~3500 EMF objects) and branch

histories up to 50 transactions; larger-scale industrial models may exhibit different performance characteristics.

Construct validity. We count blocking conflicts as the primary metric. This reflects the practical concern of reducing unnecessary user intervention during merge. A state-based tool could, in principle, be extended with heuristics to group related conflicts, but this would require domain-specific configuration that our approach avoids by leveraging the existing Reaction specifications.

7 Related Work

We discuss related work along four dimensions: model comparison and merge tools, operation-based model versioning, concurrent model synchronization in the bidirectional transformation tradition, and multi-model consistency management.

7.1 State-Based Model Comparison and Merge

EMF Compare [7, 10] is the standard Eclipse tool for state-based three-way model comparison and merge. It matches elements across model versions, computes structural diffs, and supports conflict detection and resolution. EMF Diff/Merge provides a complementary configurable merge engine. Both operate on model snapshots and treat all model differences uniformly. Our evaluation (section 6) demonstrates the consequence: when changes in derived models originate from consistency rules rather than user intent, state-based tools report false-positive conflicts. Our approach avoids this by recording operation provenance and replaying through the consistency engine.

Westfechtel [29] formalizes two-way and three-way EMF model merging with context-free and context-sensitive merge rules, proving that the merged result conforms to the metamodel. His merge ensures syntactic well-formedness of a single model but does not address semantic consistency across a network of coupled models. Our merge ensures global consistency across multiple models by replaying through *APP*.

Sharbaf and Zamani [24] present a configurable three-way merge process (E3MP) that allows domain-specific merge rules via the Epsilon platform. While E3MP supports customizable conflict detection, it operates on model snapshots and requires manual rule specification. De la Vega and Kolovos [8] propose an efficient line-based approach for resolving merge conflicts in XMI-based models, leveraging Git's merge infrastructure. Both approaches operate on individual models without awareness of cross-model consistency rules. Our approach reuses existing Vitruvius Reactions for conflict-aware merge without defining additional merge rules.

Kolovos et al. [19] introduce EML, the Epsilon Merging Language, a rule-based language for specifying model merge operations. EML combines ECL (Epsilon Comparison Language) for element matching with declarative merge and transformation rules for producing the merged result. EML can handle heterogeneous models as inputs through its flexible rule system. However, EML is fundamentally a two-way merge language: it merges a left model with a right model into a target model, with no native concept of a common ancestor or base version. Implementing three-way merge semantics requires embedding diff-against-base logic within merge rule bodies, effectively reimplementing the three-way comparison algorithm in Epsilon's expression language. EML also operates on model snapshots; like EMF Compare, it cannot distinguish whether a model element was changed by a user or derived by a consistency rule. In a multi-model consistency network, EML merge rules would report the same false-positive conflicts on derived model elements that EMF Compare does, since neither tool has access to change provenance.

Zadahmad Jafarlou and Syriani [32] present DSMCompare, a domain-specific three-way model merge tool that lifts low-level diffs to semantic differences using user-defined rules (SDRules) and visualizes conflicts in the DSL's concrete syntax. Their notion of "semantic" refers to domain-specific change patterns (e.g., recognizing a Pull-up Method refactoring from atomic changes), which is complementary to ours: we use "semantic" to denote provenance, i.e., whether a change was user-authored or produced by a consistency rule. DSMCompare operates on individual models and uses EMF Compare internally; it does not address multi-model consistency or distinguish user from derived changes. A combination of both approaches is conceivable: DSMCompare's domain-specific visualization could enhance the presentation of our provenance-aware conflict reports, while our provenance classification could inform DSMCompare's automatic resolution strategies in multi-model settings.

7.2 Operation-Based Model Versioning

Operation-based approaches record edit operations rather than comparing snapshots, providing richer information for conflict detection [6, 20].

EMFStore [18] pioneered operation-based model versioning for EMF, demonstrating that recording operations yields fewer spurious conflicts than state-based comparison. However, EMFStore tracks operations on a single model without multi-model consistency awareness. Our approach extends operation-based versioning to multi-model networks by distinguishing user-authored operations from Reaction-produced consequential changes.

Taentzer et al. [28] formalize model versioning based on graph modifications (delete/insert actions on typed attributed graphs) and provide both operation-based and state-based conflict detection with formal soundness proofs. Their dual detection validates our approach of combining changelog-based conflict detection (UUID-level, before replay) with post-replay state validation (indirect conflict detection via snapshot comparison). However, their framework operates on a single graph and does not model transformation-triggered consequential changes.

The AMOR framework [6] combines state-based and operation-based conflict detection with a taxonomy of atomic conflict patterns (update-update, delete-reference, etc.) and supports composite operation recognition. Our structural conflict detection subsumes AMOR's conflict patterns. Our novelty over AMOR is the indirect conflict class and the user-vs.-derived warning, which AMOR cannot define because it does not model consistency preservation rules.

Kehrer et al. [15] transform low-level model diffs into consistency-preserving edit scripts using higher-level operations. Their edit scripts preserve syntactic consistency of a single model. Our replay approach preserves semantic consistency across multiple models via Reaction execution.

A recent systematic mapping by Sharbaf et al. [25] analyzes 105 articles on conflict management in model merging (2001–2021). The survey identifies that syntactic conflicts are most studied and that no existing approach addresses conflicts arising from transformation-network replay. Our provenance-aware conflict classification fills this gap.

7.3 Concurrent Model Synchronization

The bidirectional transformation (BX) community studies how to reconcile concurrent updates to related models.

Xiong et al. [31] formalize concurrent update synchronization for bidirectional transformations, proving that any correct BX can be wrapped into a concurrent synchronizer. Their framework addresses two models linked by a single BX. Our work extends to networks of multiple models connected by multiple Reactions, where changes propagate transitively and create derived effects across model boundaries.

Hermann et al. [13] formalize concurrent model synchronization with triple graph grammars (TGGs), including conflict detection via element-level dependencies derived from the TGG structure. Their conflict detection is purely structural. Our three-class taxonomy adds the consequential-change dimension: we detect not only when user changes overlap, but also when Reaction-derived changes during replay overwrite user intent, a category that TGG-based approaches cannot express because they do not separate user from derived changes at the formal level.

Fritsche et al. [11] extend TGG-based concurrent synchronization with precedence-driven conflict resolution. Their precedence is defined statically on TGG rules; our precedence is defined dynamically based on runtime provenance (whether a change was user-authored or Reaction-produced).

Stevens [26] extends BX theory from binary to multiary settings, analyzing when consistency restoration can be carried out on networks of models linked by binary BXs. Our transformation network (Vitruvius Reactions linking multiple metamodels) is exactly such a network. Stevens's results characterize when network-level consistency is achievable; our contribution extends this to the branching case where consistency must be maintained across diverged and re-merged evolution paths.

Diskin et al. [9] introduce delta lenses as an algebraic framework for delta-based BXs, separating update discovery (diff) from update propagation. Our approach follows this paradigm: we extract deltas (changelogs) and propagate (replay through *APP*). The difference is that our propagation goes through the full Vitruvius pipeline including Reactions, not a single-step lens propagation.

7.4 Multi-Model Consistency Management

Klare [16] formalizes transformation networks and their correctness, proving that the orchestration problem is undecidable in the general case. Our formalization builds directly on Klare's sequential evolution semantics and extends it with branching and merge. Klare does not address divergence, common ancestors, or merge; these are our contributions.

The Vitruvius framework [17] provides the V-SUM concept, the Reactions language, and the change propagation architecture on which our implementation is built. Our work extends Vitruvius with semantic changelog capture, cross-branch conflict detection, and a replay-based merge engine.

Table 5: Comparison with related approaches.

Approach	Op.-based	Multi-model	Provenance
EMF Compare [10]	×	×	×
EMFStore [18]	✓	×	×
AMOR [6]	✓	×	×
EML [19]	×	×‡	×
DSMCompare [32]	×	×	×†
TGG sync [13]	✓	×*	×
Ours	✓	✓	✓

*TGG-based synchronization handles two models linked by one grammar.
†DSMCompare lifts diffs to domain-specific semantic patterns but does not distinguish user-authored from consistency-rule-derived changes.
‡EML supports heterogeneous model inputs via custom rules but has no concept of coupled models with consistency preservation; it is also limited to two-way merge.
Op.-based = uses operation histories; Multi-model = handles networks of >2 coupled models;
Provenance = distinguishes user-authored from derived changes for conflict classification.

Pascual et al. [22] provide formal foundations for consistency in model-driven development, defining consistency as a relation over model tuples. Our consistency relation *CR* follows their definition; our merge operator extends it to the branching case.

Stünkel et al. [27] introduce comprehensive systems as a category-theoretical construction for representing multiary consistency relations among multiple models. They show that not every multiary relation decomposes into binary ones. Our V-SUM is a comprehensive system in their sense; our directional merge operates on the full multiary state via *APP*, rather than attempting pairwise binary merges that could lose global consistency.

Barkowsky and Giese [4] introduce multi-version models as a compact encoding of multiple model versions in a single graph, enabling simultaneous well-formedness checking and conflict detection. Their approach does not consider consistency-preserving transformations; our approach maintains separate branches and merges them by replaying changes through transformation rules.

7.5 Connection to Database Concurrency Control

Our interleaving merge (section 4.11) draws on the notion of conflict serializability from database concurrency theory [5]. In the database setting, two transactions conflict if they access the same data item and at least one writes; a cycle in the precedence graph means no serial schedule is equivalent. Our footprint dependency graph adapts this idea to the model-transformation domain: instead of database read/write conflicts, we detect overlaps between a commit's *consequential* (Reaction-derived) footprint and another commit's *direct* (user-authored) footprint. This adaptation is necessary because, unlike database transactions, our transactions produce derived side effects through consistency rules, and it is these derived effects that create ordering constraints. The cycle-detection step uses Kahn's BFS-based topological sort [14].

7.6 Summary

Table 5 positions our approach against the most relevant related work.

8 Conclusion and Future Work

We presented a directed, replay-based three-way merge for networks of coupled models maintained by consistency preservation rules. The idea is to record user-intended change transactions per commit and replay them through the consistency preservation engine, so that derived changes are regenerated rather than merged. This enables a provenance-aware conflict classification that distinguishes direct conflicts between user actions, warnings when user intent overrides derived state, and indirect conflicts when Reaction-derived changes threaten user intent on the target branch. A bidirectional fallback attempts the reverse merge direction when indirect conflicts arise. When both directions produce indirect conflicts in disjoint model areas, a dependency-graph-based interleaving algorithm finds an ordering of commits from both branches that avoids all indirect conflicts.

We formalized the merge operator, including the interleaving merge and its footprint dependency graph (section 4.11), showed that successful merges preserve global consistency, and implemented the approach as an extension to the Vitruvius framework. An evaluation on two three-model case studies from different automotive domains (brake system engineering and automated driving) shows that, across 14 hand-crafted scenarios, our approach reports two blocking conflicts where EMF Compare reports 26. The 24 additional EMF Compare conflicts are on derived model elements that our approach handles as non-blocking warnings. The conflict reduction is consistent across both domains (9 and 15 fewer blocking conflicts, respectively). Four additional interleaving scenarios demonstrate that the algorithm resolves cross-area conflicts that neither the forward nor reverse direction can handle individually.

Limitations and future work. The current approach has several limitations that suggest directions for future work. First, the interleaving algorithm's reaction-footprint estimates are deliberately conservative: footprint sets grow monotonically and never shrink, which guarantees termination but may occasionally report `INTERLEAVING_CONFLICT` when a valid ordering does in fact exist. A more precise algorithm could track footprint shrinkage (e.g., when elements are deleted before a commit is replayed), but this substantially complicates the analysis. Second, the footprint model tracks only direct element-feature writes, not cascade effects of EMF containment. When an element is deleted, EMF implicitly removes all its contained children. If another commit's changelog references a child element by HierarchicalId, the footprints may not overlap (the parent UUID differs from the child UUID), yet replay fails because the child no longer exists. The current implementation handles this via guard-failure fallback (section 5.5, Phase 3), but static detection of such containment-induced dependencies remains future work. Third, when an indirect conflict is detected, an alternative to the bidirectional fallback could be to *reorder* the conflicting transaction so that the user's explicit choice takes precedence over the derived state. Investigating under which conditions such reordering preserves consistency is an open problem. Fourth, the current conflict classification does not cover *replay divergence*: a case where replay succeeds operationally but the consequential changes differ materially from those produced on the source branch. Detecting such silent semantic deviations is an open problem. Fifth, the formalization assumes a fixed Reaction set across both branches. If Reactions themselves evolve on different branches, the merge must also reconcile the transformation rules, which introduces additional complexity. Sixth, the evaluation covers two case study domains with three models each. While both are from the automotive sector, they use different metamodel structures and Reaction topologies. Evaluating on larger-scale industrial models, longer branch histories, and non-automotive domains would further strengthen the generalizability claims. Finally, the current implementation requires programmatic conflict resolution. Integrating an interactive resolution interface, potentially building on domain-specific visualization approaches such as DSMCompare [32], and studying how the provenance-aware warnings affect user decision-making in practice are important next steps.

References

[1] Rasmus Adler, Marcel Klose, Peter Munk, Clemens Braunstein, and Markus Lipaczewski. 2019. A Dataset for Model-Based Safety Assurance of Automated Driving Systems. In *Workshop on Autonomous Systems Design (ASD) (OpenAccess Series in Informatics (OASIcs), Vol. 74)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 4:1–4:8. doi:10.4230/OASIcs.ASD.2019.4

[2] Kerstin Altmanninger, Martina Seidl, and Manuel Wimmer. 2009. A survey on model versioning approaches. *International Journal of Web Information Systems* 5, 3 (2009), 271–304. doi:10.1108/17440080910983556

[3] Colin Atkinson and Thomas Kühne. 2003. Model-driven development: A metamodeling foundation. *IEEE Software* 20, 5 (2003), 36–41. doi:10.1109/MS.2003.1231149

[4] Matthias Barkowsky and Holger Giese. 2022. Towards development with multi-version models: Detecting merge conflicts and checking well-formedness. In *Graph Transformation (ICGT 2022) (Lecture Notes in Computer Science, Vol. 13349)*. Springer, 118–136. doi:10.1007/978-3-031-09843-7_7

[5] Philip A. Bernstein, Vassos Hadzilacos, and Nathan Goodman. 1987. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley.

[6] Petra Brosch, Gerti Kappel, Philip Langer, Martina Seidl, Konrad Wieland, and Manuel Wimmer. 2012. An introduction to model versioning. In *Intl. School on Formal Methods for the Design of Computer, Communication and Software Systems (SFM)*. LNCS, Vol. 7320. Springer, 336–398. doi:10.1007/978-3-642-30982-3_10

[7] Cédric Brun and Alfonso Pierantonio. 2008. Model differences in the Eclipse modeling framework. *UPGRADE, The European Journal for the Informatics Professional* 9, 2 (2008), 29–34.

[8] Alfonso de la Vega and Dimitris S. Kolovos. 2022. An efficient line-based approach for resolving merge conflicts in XMI-based models. *Software & Systems Modeling* 21, 6 (2022), 2461–2487. doi:10.1007/s10270-022-00976-4

[9] Zinovy Diskin, Yingfei Xiong, and Krzysztof Czarnecki. 2011. From state- to delta-based bidirectional model transformations: The symmetric case. In *Intl. Conf. on Model Driven Engineering Languages and Systems (MoDELS) (LNCS, Vol. 6981)*. Springer, 304–318. doi:10.1007/978-3-642-24485-8_22

[10] Eclipse Foundation. 2024. *EMF Compare*. https://eclipse.dev/emf/compare/

[11] Lars Fritsche, Jens Kosiol, Adrian Möller, Andy Schürr, and Gabriele Taentzer. 2020. A precedence-driven approach for concurrent model synchronization scenarios using triple graph grammars. In *Proc. 13th ACM SIGPLAN Intl. Conf. on Software Language Engineering (SLE)*. ACM, 39–55. doi:10.1145/3426425.3426931

[12] Nathan Hagel, Johannes Mäkelburg, Claus Hammann, Thomas Weber, Thomas Alexander Völk, Francesco P. Urbano, Patrick Grycz, Katharina Bause, Minakshi Kaushik, Vincenzo Scotti, Akhila Bairy, Maike Schwammberger, Maribel Acosta, Albert Albers, Anne Koziolek, and Tobias Düser. 2025. Explainability in Automated Cross-Domain Model-Driven Brake System Development. In *2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*. 204–211. doi:10.1109/ASEW67777.2025.00047

[13] Frank Hermann, Hartmut Ehrig, Claudia Ermel, and Fernando Orejas. 2012. Concurrent model synchronization with conflict resolution based on triple graph grammars. In *Intl. Conf. on Fundamental Approaches to Software Engineering (FASE) (LNCS, Vol. 7212)*. Springer, 178–193. doi:10.1007/978-3-642-28872-2_13

[14] Arthur B. Kahn. 1962. Topological Sorting of Large Networks. *Commun. ACM* 5, 11 (1962), 558–562. doi:10.1145/368996.369025

[15] Timo Kehrer, Udo Kelter, and Gabriele Taentzer. 2013. Consistency-preserving edit scripts in model versioning. In *Proc. 28th IEEE/ACM Intl. Conf. on Automated Software Engineering (ASE)*. IEEE, 191–201. doi:10.1109/ASE.2013.6693079

[16] Heiko Klare. 2022. *Building Transformation Networks for Consistent Evolution of Interrelated Models*. Vol. 34. KIT Scientific Publishing. doi:10.5445/KSP/1000150511

[17] Heiko Klare, Max E. Kramer, Michael Langhammer, Dominik Werle, Erik Burger, and Ralf Reussner. 2021. Enabling consistency in view-based system

development—the Vitruvius approach. *Journal of Systems and Software* 171 (2021), 110815. doi:10.1016/j.jss.2020.110815

[18] Maximilian Koegel and Jonas Helming. 2010. EMFStore: A model repository for EMF models. In *Proc. 32nd ACM/IEEE Intl. Conf. on Software Engineering (ICSE)*. 307–308. doi:10.1145/1810295.1810364

[19] Dimitrios S. Kolovos, Richard F. Paige, and Fiona A.C. Polack. 2006. Merging models with the Epsilon Merging Language (EML). In *Intl. Conf. on Model Driven Engineering Languages and Systems (MoDELS) (LNCS, Vol. 4199)*. Springer, 215–229. doi:10.1007/11880240_16

[20] Tom Mens. 2002. A state-of-the-art survey on software merging. *IEEE Transactions on Software Engineering* 28, 5 (2002), 449–462. doi:10.1109/TSE.2002.1000449

[21] Richard F. Paige, Nicholas Matragkas, and Louis M. Rose. 2016. Evolving models in model-driven engineering: State-of-the-art and future challenges. *Journal of Systems and Software* 111 (2016), 272–280. doi:10.1016/j.jss.2015.08.047

[22] Romain Pascual, Bernhard Beckert, Mattias Ulbrich, Michael Kirsten, and Wolfram Pfeifer. 2024. Formal Foundations of Consistency in Model-Driven Development. In *12th Intl. Symp. on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA 2024) (Lecture Notes in Computer Science, Vol. 15222)*. Springer, 178–200. doi:10.1007/978-3-031-75380-0_11

[23] Ralf Reussner, Ina Schaefer, Bernhard Beckert, Anne Koziolek, and Erik Burger. 2023. Consistency in the View-Based Development of Cyber-Physical Systems (CONVIDE). In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 83–84. doi:10.1109/MODELS-C59198.2023.00024

[24] Mohammadreza Sharbaf and Bahman Zamani. 2020. Configurable three-way model merging. *Software: Practice and Experience* 50, 8 (2020), 1565–1599. doi:10.1002/spe.2835

[25] Mohammadreza Sharbaf, Bahman Zamani, and Gerson Sunyé. 2023. Conflict management techniques for model merging: A systematic mapping review. *Software & Systems Modeling* 22 (2023), 1031–1079. doi:10.1007/s10270-022-01050-9

[26] Perdita Stevens. 2020. Maintaining consistency in networks of models: Bidirectional transformations in the large. *Software & Systems Modeling* 19, 1 (2020), 39–65. doi:10.1007/s10270-019-00736-x

[27] Patrick Stünkel, Harald König, Yngve Lamo, and Adrian Rutle. 2021. Comprehensive Systems: A formal foundation for Multi-Model Consistency Management. *Formal Aspects of Computing* 33, 6 (2021), 1067–1114. doi:10.1007/s00165-021-00555-2

[28] Gabriele Taentzer, Claudia Ermel, Philip Langer, and Manuel Wimmer. 2014. A fundamental approach to model versioning based on graph modifications: From theory to implementation. *Software & Systems Modeling* 13, 1 (2014), 239–272. doi:10.1007/s10270-012-0248-x

[29] Bernhard Westfechtel. 2014. Merging of EMF models: Formal foundations. *Software & Systems Modeling* 13, 2 (2014), 757–788. doi:10.1007/s10270-012-0279-3

[30] Claes Wohlin, Per Runeson, Martin Höst, Magnus C Ohlsson, Björn Regnell, Anders Wesslén, et al. 2012. *Experimentation in software engineering*. Vol. 236. Springer.

[31] Yingfei Xiong, Hui Song, Zhenjiang Hu, and Masato Takeichi. 2013. Synchronizing concurrent model updates based on bidirectional transformation. *Software & Systems Modeling* 12, 1 (2013), 89–104. doi:10.1007/s10270-010-0187-3

[32] Manouchehr Zadahmad Jafarlou and Eugene Syriani. 2026. Domain-specific conflict resolution and model merge. *Journal of Systems and Software* 232 (2026), 112694. doi:10.1016/j.jss.2025.112694